

KONTROL İŞLEMLERİ

Kontrol İşlemleri

Kontrol işlemleri (**control operation**) veya **kontrol akışı** (**control flow**) programın icra sırasını değiştiren kontrol yapıları olup bu **talimatlar** (**statement**) olup üç türü vardır;

1. **Duruma göre seçimler** (**conditional choice**): if, if-else ve match talimatları.
2. **İlişkisel döngü** (**relational loop**): for, for-else ve while talimatları.
3. **Dallanmalar** (**jump**): Fonksiyon çağrılar ve continue, break, pass ve return talimatları.

Kontrol işlemlerinde; kodlana mantıksal satırlar (**logical sequence**) ile fiziksel olarak icra edilen satırlar (**physical sequence**) birbirinden farklıdır.

Satırlar ve Girintileme

Python programlama, sınıf (**class**), fonksiyon (**function**) tanımları veya kontrol işlemlerinde (**control operation**) kullanılacak kod blokları için tırnaklı parantez kullanmaz. C/C++/C# ve Java dillerinde kod bloğu tırnaklı parantezle başlar ve biter. Ancak Python dilinde kod blokları, katı bir şekilde uygulanan **satır girintisi** (**indentation**) ile yapılır.

💡 Standart olarak 4 boşluk karakteri ile girinti verilir!

Girintileme, iki nokta üst üste (:) karakteri kod bloğunu başlatır. Bu karakterden sonra talimatlar girintili yazılır. Girinti bittiğinde kod bloğu bitmiş sayılır.

```
yaş=150
if yaş>120: # If koşulu doğrulanırsa icra edilecek blok buradan başlıyor
    print('Hatalı Yaş Değeri:',yaş) # Bu satır blok içinde
    yaş=120 # Bu satır da girintili yazıldığından bu satırda aynı blok içindedir.
print(yaş)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Hatalı Yaş Değeri: 150
120
```

Yukarıdaki örnekte if koşulundan sonraki iki nokta üst üste karakterinden sonra bir alt satıra geçildiğinde 4 karakter girintileme yapılmıştır. Girintileme bir alt satırda da devam etmiştir. Bu durumda if koşulu gerçekleşirse bu iki satır icra edilecektir. Bir başka örnek aşağıda verilmiştir;

```
if True:
    print ("Doğru") #Doğru ekrana basılır
else:
    print ("Yanlış")
Doğru

if True:
    print ("Cevap") #Cevap
    print ("Doğru") #Doğru
else:
    print ("Cevap")
    print ("Yanlış")
```

Eğer tek talimat yazılacak ise aşağıdaki gibi girintileme yapılmayabilir;

```
yaş=150
if yaş>120: print('Hatalı Yaş Değeri!') #Hatalı Yaş Değeri! Ekrana basılır
else: print('Yaş:',yaş)
```

Ayrıca Python dilinde eğer bir **talimat** (**statement**) birden fazla satırda yazılacak ise ters bölü karakteri ile birden fazla satırda yazılabilir. Bu karakterden sonra talimatın devamına alt satırda girintili bir şekilde devam edilir.

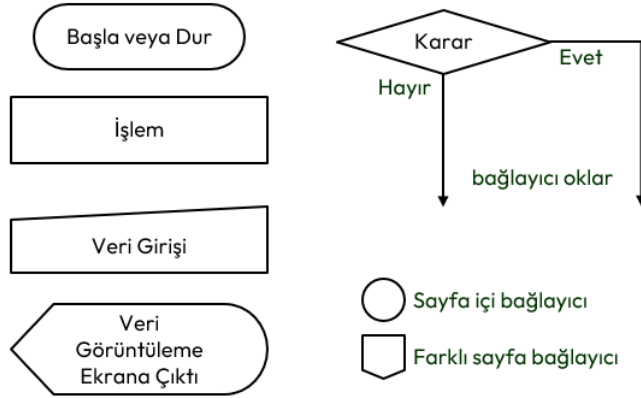
```
birincisatış = 10000
ikincisatış = 30000
üçüncüsatış = 40000
toplam = birincisatış + \
        ikincisatış + \
        üçüncüsatış
print(toplam) #80000
```

[], {} veya () parantezleri içinde yer alan ifadelerin satır devam karakterini kullanması gerekmez.

```
günler=['Pazartesi', 'Salı',
        'Çarşamba', 'Perşembe',
        'Cuma']
```

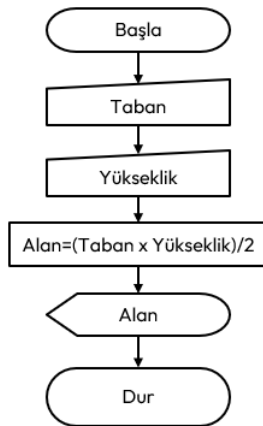
Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.



Şekil 6. Akış Diyagramları Genel Şekilleri

Akış diyagramları (**flowchart**), adımların grafiksel gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.



Şekil 7. Örnek Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- Akış şemalarında tek bir başlangıç simgesi olmalıdır
- Bitiş simgesi birden çok olabilir.
- Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

Aşağıda taban ve yüksekliği klavyeden girilen bir üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.

Bu akış diyagramına ilişkin program aşağıda verilmiştir;

```
tabanstr=input('Tabanı Giriniz:')
yükseklikstr=input('Yüksekliği Giriniz:')
alan=float(tabanstr)*float(yükseklikstr)/2
print(alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Tabanı Giriniz:3.5
Yüksekliği Giriniz:4
7.0
```

Duruma Göre Seçimler

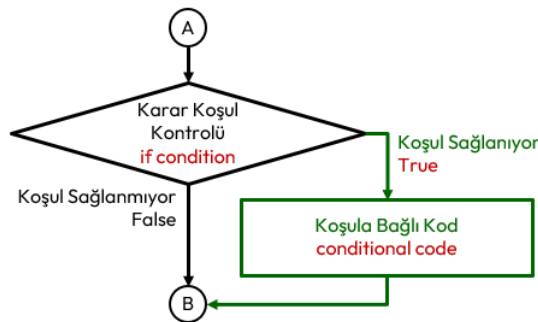
Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre değiştiren talimatlardır.

If Talimatı

If talimatı, **karar vermeye** (**decision making**) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. Mutlaka Boolean bir ifade olan **koşul** (**condition**) ifadesi test edilir sonuç **True** ise **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez.

Sözde kodu aşağıda verilmiştir.

```
if koşul:
    koşul-kodu
```



Şekil 8. If Talimatı İcra Akışı

If talimatı içeren bir kodun icrası;

		yaş	cinsiyet
1	yaş=18	18	?
2	cinsiyet='K'	18	'K'
3	if yaş<30:	18	'K'
4	print("Genç")	18	'K'
5	if cinsiyet=='E':	18	'K'
	print("Erkek")	18	'K'

Tablo 15. If Talimatı İçeren Bir C Programının İcrası

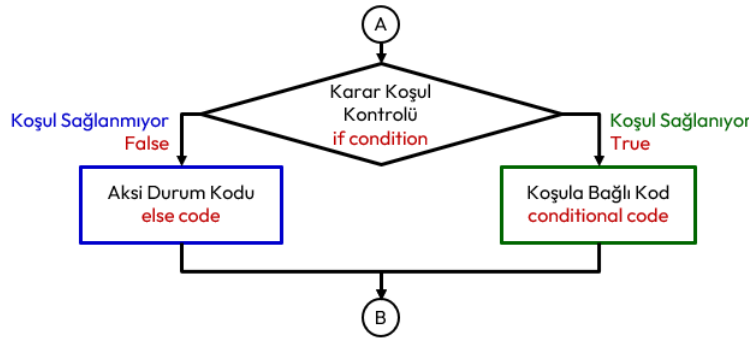
Programın icrasında neler olduğu aşağıda verilmiştir;

1. yaş değişkenine 18 değeri atanır.
2. cinsiyet değişkenine 'K' atanır.
3. if talimatındaki **koşul** (condition) test edilir. `yas<30` yani `18<30` testinde sonuç **True** olduğundan izleyen **koşul kodu** (conditional code) icra edilir.
4. `print("Genç")` koşul kodu olduğundan icra edilir.
5. if talimatındaki koşul test edilir. `cinsiyet=='E'` yani `'E'=='K'` testinde sonuç **False** olduğundan izleyen koşul kodu icra edilmez.

If-Else Talimatı

If-Else Talimatı, bir başka **karar verme** (decision making) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatına benzer şekilde **koşul** (condition) ifadesi True ise **koşul kodu** (conditional code) icra edilir, değilse **aksi durum kodu** (else code) icra edilir.

```
if koşul:
    koşul-kodu
else:
    aksi-durum-kodu
```



Şekil 9. If-Else Talimatı Sözde Kodu ve İcra Akışı

Bu talimata ilişkin örnek aşağıda verilmiştir;

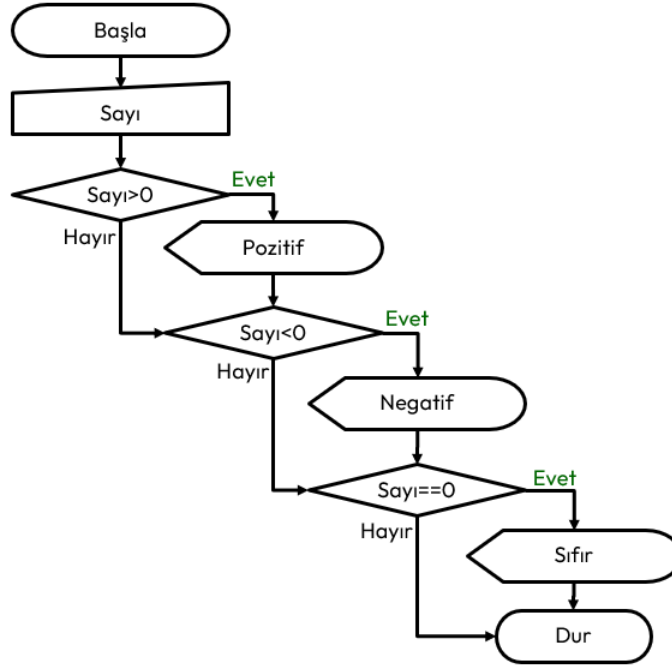
```
yaş=18
cinsiyet='K'
if yaş<30: # yaş test ediliyor
    print("Genç") #burada yaş değerinin 30 dan küçük olduğu biliniyor. Genç Yazılır
else:
    print("Yaşlı") #burada yaş değerinin 30 dan küçük olmadığı biliniyor
if cinsiyet=='E': #cinsiyet test ediliyor
    print("Erkek") #burada cinsiyet değerinin 'E' olduğu biliniyor
else:
    print("Kadın") #burada cinsiyet değerinin 'E' olmadığı biliniyor. Kadın Yazılır
```

If talimatında test edilen değerler **elif** eklenerek çoğaltılabilir. Bu durumda her eklenen **elif** için ayrı koşul kodu yazılır;

```
yaş=18
cinsiyet='K'
if yaş<30:
    print("Genç") # bu blokta yaş değerinin 30 dan küçük olduğu biliniyor
elif yaş <60:
    print("Orta Yaşlı") # bu blokta yaş değerinin 30 dan küçük olmadığı yani
                        # önceki testten geçmediği ve 60 den küçük olduğu
                        # biliniyor
else:
    print("Yaşlı") # bu blokta yaş değerinin önceki iki testten geçmediği
                  # ve 60 den küçük olmadığı biliniyor
if cinsiyet=='E': #cinsiyet test ediliyor
    print("Erkek") #burada cinsiyet değerinin 'E' olduğu biliniyor
else:
```

```
print("Kadın") #burada cinsiyet değerinin 'E' olMAdığı biliniyor
```

Örnek olarak klavyeden girilen bir sayının sıfırdan küçük mü? Büyük mü? ya da sıfıra eşit mi? Olduğunu bulan bir programı yazalım. Programın icra akışı aşağıdaki şekilde olacaktır.



Şekil 10. Programın If Talimatlarıyla Yazılması Halinde İcra Akışı

Bu akışı çalıştıran program kodu aşağıda verilmiştir;

```
sayistr=input("Bir Tamsayı Giriniz:")
sayı=int(sayistr)
if sayı>0:
    print("Pozitif")
if sayı<0:
    print("Negatif")
if sayı==0:
    print("Sıfır")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Tamsayı Giriniz:12
Pozitif
```

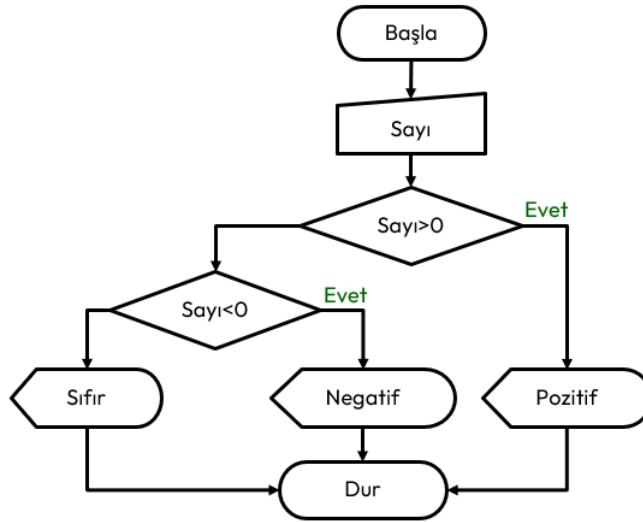
İcra akışı incelendiğinde ilk **if** talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmaz. Benzer şekilde ilk iki test geçilirse üçüncü **if** talimatındaki teste gerek yoktur. Aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```
sayistr=input("Bir Tamsayı Giriniz:")
sayı=int(sayistr)
if sayı>0:
    print("Pozitif")
elif sayı<0:
    print("Negatif")
else:
    print("Sıfır")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Tamsayı Giriniz:12
Pozitif
```

Bu durumu aşağıdaki akış diyagramında daha net görebiliriz.



Şekil 11. Programın If-Else Talimatlarıyla Yazılması Halinde İcra Akışı

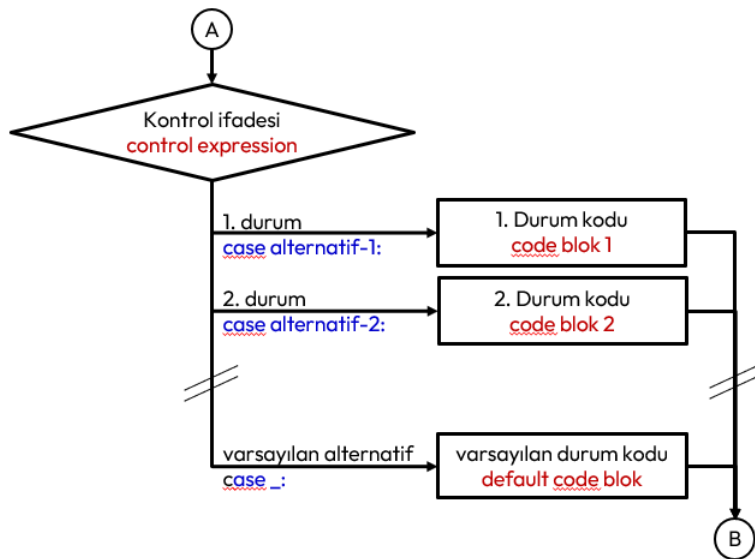
Program incelendiğinde sayının pozitif girilmesi halinde sadece ilk **if** talimatındaki test geçecek ve sonrasındaki aksi durum kodlarına ilişkin bloklar icra edilmeyecektir. Sayının negatif girilmesi halinde **elif** bloğuna girilecek ve sonrasındaki **else** kısmı çalıştırılmayacaktır. Sayı sıfır girilmiş ise **else** kodu çalıştırılacaktır.

Match Talimatı

Seçme (**match**) talimatı, bir kontrol **ifadesi** (**expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Alternatifler, kontrol ifadesini seçen **desenlerdir** (**pattern**). Sözde kodu aşağıda verilmiştir;

```

match kontrolifadesi:
    case alternatif-1:
        alternatif-1-kodu
    case alternatif-2:
        alternatif-2-kodu
    # ...
    case _:
        varsayılan-alternatif-kodu
  
```



Şekil 12. Switch Talimatı Sözde Kodu ve İcra Akışı

⚠ match-case yapısı Python 3.10 ve üzeri sürümlerde kullanılabilir. Eski sürümlerde **if-elif-else** kullanılmalıdır.

Seçim talimatı, bir ifadeyi alır ve değerini bir veya daha fazla durum bloğu olarak verilen ardışık desenlerle karşılaştırır. İfadeye uyan desendeki durum kodu icra edilir. `match` talimatına ilişkin kurallar;

1. **Kontrol ifadesi** (**control expression**), sonucu tamsayı olan ifadedir ve bir kez değerlendirilir.
2. Bloğu olmayan bir `match` talimatı yazılamaz!
3. Tüm alternatifleri içerecek şekilde alternatiflerin tümü yazılmayabilir.
4. Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için `_`: etiketli alternatif yazılabilir.

Örnek program aşağıda verilmiştir;

```
gün=input("Gün Giriniz:")
match gün:
    case "Pazartesi" | "Salı" | "Çarşamba" | "Perşembe": #birden fazla desen
        print("Girdiğiniz Gün:{}".format(gün))
    case "Cuma": print("Mübarek Gün.") #tek desen
    case "Cumartesi": print("Yarım Gün Çalışma.") #tek desen
    case "Pazar": print("Tatil.") #tek desen
    case _: print("Geçerli bir gün adı Girmediniz!") #yukarıdaki desenlere uymayanlar
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Gün Giriniz:Cuma
Mübarek Gün
```

Seçim talimatı ifadeyi herhangi bir sabitle eşleştirmez. Bir listeyi durum değeri olarak kullanılabilir.

```
liste=["İyi Çalışmalar", "İlhan"] #iki elemanlı bir liste
match liste:
    case [selamlama,isim]: # her bir eleman bir değişkenile işlenebilir.
        print(selamlama,"!",isim)

hesaplistesi=[True,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu Yevmiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)

hesaplistesi=[False,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu yevbiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İyi Çalışmalar ! İlhan
Haftasonu Yevmiyesi: 2000
Hafta İçi Yevmiyesi: 1600.0
```

Ayrıca, listedeki değişken sayıda öge için, yıldız (*) operatörüyle bir diziye ayrıştırılabilirler.

```
liste=["İlhan","Pazartesi","Çarşamba","Perşembe","Cuma","Cumartesi"] # 6 elemanlı liste

match liste:
    case [isim,*günler]: #ikiden fazla elemanlı liste girilirse ilk elemandan sonrası
                        #diziye ayrıştırılıyor.
        print(isim," Şu günler Çalıştı:")
        for gün in günler:
            print(gün,end=",")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İlhan Şu günler Çalıştı:
Pazartesi,Çarşamba,Perşembe,Cuma,Cumartesi,
```

Bir başka örnek;

```
liste=["İlhan","Pazartesi"]
match liste:
    case [isim,gün]: # Eğer iki elemanlı bir liste girilirse
        print(isim,",", gün, " Günü çalıştı")
    case [isim,*günler]: # İki'den fazla elemanlı liste girilirse
        print(isim," Şu günler çalıştı:")
        for gün in günler:
            print(gün,",")
```

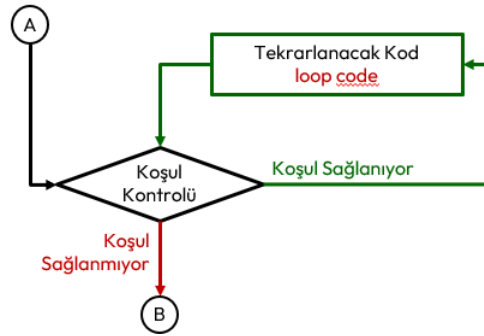
Örneğin program akışı aşağıdaki gibi olacaktır.

İlhan , Pazartesi Günü çalıştı

İlişkisel Döngüler

Döngüler (loop) bir ifadeyi veya ifadeler grubunu birden çok kez yürütmemize olanak tanır. Bunu yapmak için tekrarlanacak kodun başına gitmek için eskiden GOTO talimatı kullanılırdı. Bu talimatın kullanılması kodun okunaklılığını azaltan bir şeydir. GOTO 1968 Yılında *Edsger W. Dijkstra* GOTO/JUMP TO ifadelerini zararlı olarak ilan edilmiştir¹². GOTO kullanmamak için; **for**, **for-next** ve **while** gibi **kontrol talimatları** (control statement) yapısal programlamaya eklenmiştir.

Python dili de yapısal programlamayı destekleyen bir dildir.

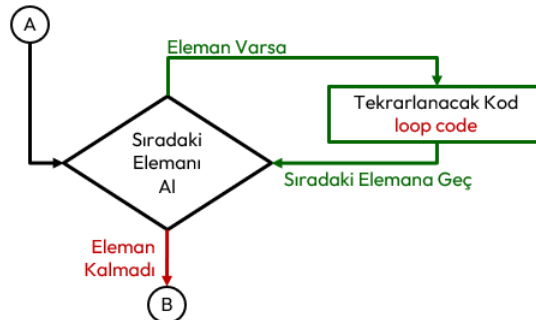


Şekil 13. Bir Döngünün İcra Akışı

For Talimatı

Python dilindeki **for** döngüsü, bir liste, **demet** (tuple) veya bir **dizgi** (string) gibi herhangi bir dizinin elemanları üzerinde döngü yapma yeteneği sağlar. Dizinin her elemanına aynı eylemi gerçekleştirir.

```
for yineleme-elemanı-değişkeni in yilenebilir-yapı:
    döngü-kodu
```



Şekil 14. For Talimatı İcra Akışı

¹² <https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

Örnek program aşağıda verilmiştir;

```
metin="Merhaba!"
for karakter in metin:
    print(karakter)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
M
e
r
h
a
b
a
!
```

Bir liste üzerinde for döngüsüne örnek;

```
satışlar=[10000,12000,10000,5000] #liste üzerinde for döngüsü
toplam=0
for satış in satışlar:
    toplam +=satış
print("Satış Toplamı",toplam)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Satış Toplamı 37000
```

Tek sayılardan oluşan demet (tuple) üzerine for döngüsü

```
ondanküçükteksayılar=(1,3,5,7,9)
for teksayı in ondanküçükteksayılar:
    print(teksayı,end="-")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
1-3-5-7-9-
```

Sözlük üzerinde for döngüsü

```
sözlük={10:"On", 20:"Yirmi", 30:"Otuz"}
for eleman in sözlük:
    print(eleman)
for eleman in sözlük:
    print(eleman,"-", sözlük[eleman])
for eleman in sözlük.items():
    print(eleman)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
10
20
30
10 - On
20 - Yirmi
30 - Otuz
(10, 'On')
(20, 'Yirmi')
(30, 'Otuz')
```

Aralık Nesneleri

Dahili range() fonksiyonu, bir tamsayı dizisi oluşturarak bu dizi üzerinde bir yineleyici (iterable) nesnesi döndürür. Bu aralık (range) nesnesi, adım parametresiyle ayrılmış, başlangıçtan bitişe kadar tamsayılar içerir.

Sözde kodu aşağıda verilmiştir.

```
range (başlangıç, bitiş, adım)
```

Burada **başlangıç**, aralığın başlangıç değeri. İsteğe bağlı olup varsayılan değeri 0'dır. **bitiş** değerini almayacak son elemana kadar dizi oluşturulur. **adım** kadar dizi elemanlarının arasındaki değer farkıdır. **adım** için varsayılan değer 1'dir. Aşağıda örnek program verilmiştir;

```
aralık=range(4)
for sayı in aralık:
    print(sayı,end=',') # end-ayraç (separator) anlamındadır
print()
aralık=range(1,5)
for sayı in aralık:
    print(sayı,end='-') # end-ayraç (separator) anlamındadır
print()
aralık=range(10,20,2)
for sayı in aralık:
    print(sayı,end=':') # end-ayraç (separator) anlamındadır
```

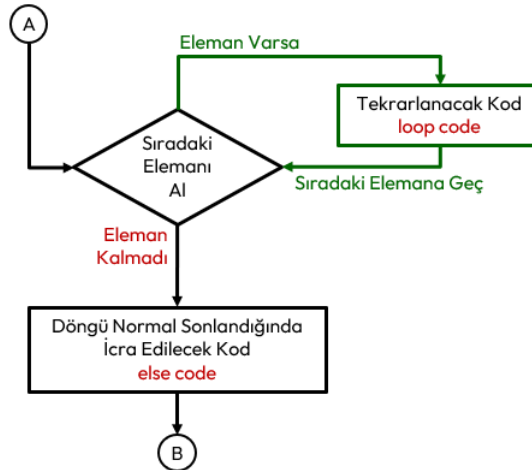
Örneğin program akışı aşağıdaki gibi olacaktır.

```
0,1,2,3,
1-2-3-4-
10:12:14:16:18:
```

For-Else Talimatı

Python, bir **for** döngüsüyle ilişkilendirilecek isteğe bağlı bir **else** bloğunu destekler. Bu **else** bloğu bir **for** döngüsüyle kullanılırsa, yalnızca **for** döngüsü normal şekilde sonlandığında icra edilir. Aşağıda sözde kodu verilmiştir.

```
for değişken in yinelenenebilir-yapı:
    #tekrarlanacak talimatlar
else:
    #döngü normal bir şekilde sonlandığında icra edilecek talimatlar
```



Şekil 15. For-Else Talimatı İcra Akışı

Aşağıda örnek bir program verilmiştir;

```
for sayaç in range(5):
    print('{}. Adım...'.format(sayaç))
else:
    print('For Döngüsü Bitti.!')
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0. Adım...
```

```

1. Adım...
2. Adım...
3. Adım...
4. Adım...
For Döngüsü Bitti.!

```

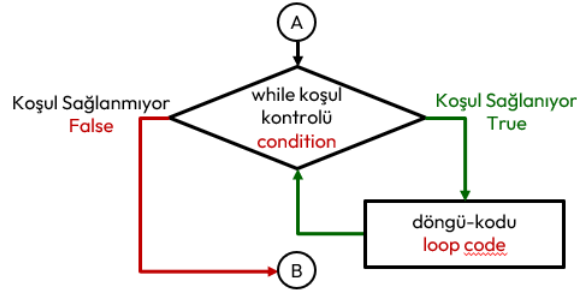
While Talimatı

While döngüsünde, **koşul** (**condition**) kontrolü **True** olduğu sürece **döngü kodu** (**loop code**) icra edilir. Koşul herhangi bir ifade olabilir ve **True** sıfır olmayan herhangi bir değerdir. İfade **False** olur olmaz, program kontrolü döngü dışındaki izleyen talimatla devam eder. Sözde kodu aşağıda verilmiştir;

```

while koşul-ifadesi:
    döngü-kodu

```



Şekil 16. While Talimatı İcra Akışı

Örnek uygulama aşağıda verilmiştir.

```

sayaç=1
while sayaç<=10:
    print(sayaç, ".Adım")
    sayaç+=1

```

Örneğin program akışı aşağıdaki gibi olacaktır.

```

1 .Adım
2 .Adım
3 .Adım
4 .Adım
5 .Adım
6 .Adım
7 .Adım
8 .Adım
9 .Adım
10 .Adım

```

İç İçe Döngü Kodu

Döngüler içinde yer alan **döngü kodu** (**loop code**) bir başka döngüyü içerebilir. **İç içe döngülerde** (**nested loop**), bir döngü diğerinin içinde tanımlanır. Başka bir döngüyü yani iç döngüyü çevreleyen döngüye **dış döngü** (**outer loop**) denir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü kullanırız. Aşağıda 3 satır 10 adet yıldız basan bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci **for** döngüsüdür.

```

for satır in range(1,4):
    for sütun in range(1,10):
        print('*',end='')
    print() #alt satıra geçmek için.

```

Örneğin program akışı aşağıdaki gibi olacaktır.

```

*****
*****
*****

```

Aşağıda elemanları, satır ve sütun indislerinin çarpımı olan bir 4x5 boyutlu bir matris ekrana yazdırılmıştır;

```
for satır in range(1,5):
    for sütun in range(1,6):
        print('{0:<4}'.format(satır*sütun), end=' ')
    print()
```

Örneğin program akışı aşağıdaki gibi olacaktır.

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20

Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
sayı=-1 # while döngüsüne girmek için -1 olarak ayarlandı. 0 olsaydı girilmezdi!
toplam=0
while sayı!=0: # sentinel value 0
    sayıstr=input('Sayı Giriniz (Bitirmek İçin 0):')
    sayı=int(sayıstr)
    toplam+=sayı
print(toplam)
```

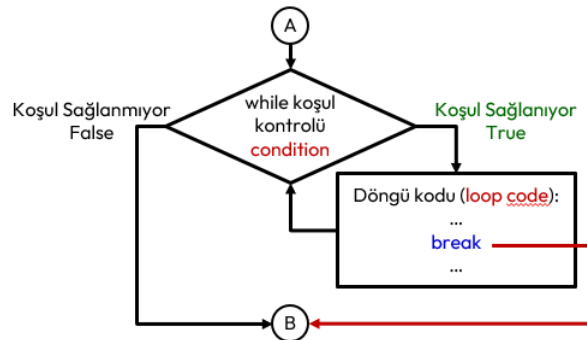
Örneğin program akışı aşağıdaki gibi olacaktır.

```
Sayı Giriniz (Bitirmek İçin 0):10
Sayı Giriniz (Bitirmek İçin 0):30
Sayı Giriniz (Bitirmek İçin 0):40
Sayı Giriniz (Bitirmek İçin 0):50
Sayı Giriniz (Bitirmek İçin 0):0
130
```

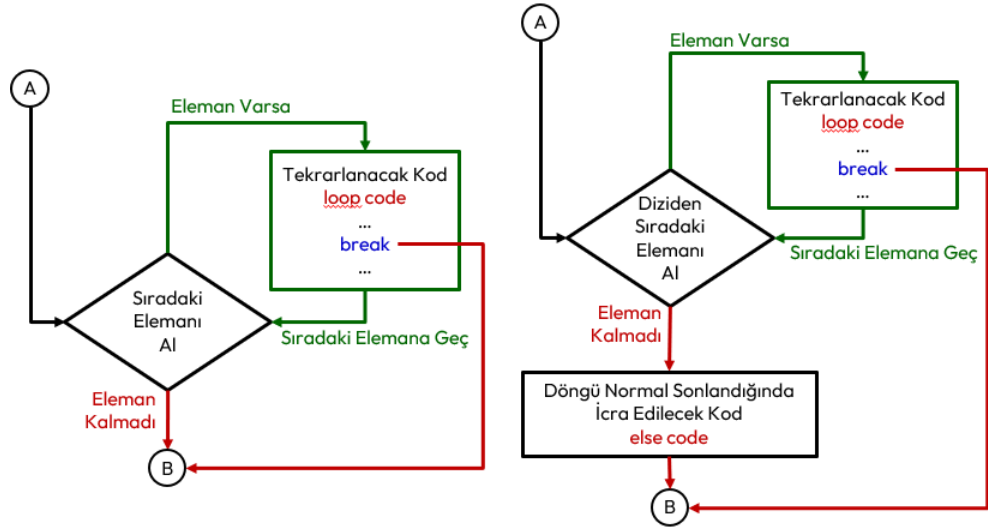
Dallanmalar

Break Talimatı

C dilindeki geleneksel **break** talimatı gibi geçerli döngüyü sonlandırarak döngü sonrasındaki ilk talimatı icra etmek için kullanılır. Bu ifade, bir döngüden ani bir çıkış gerektiren bazı koşullarda kullanılır. Hem **while** hem de **for** döngülerinde kullanılabilir.



Sekil 17. While Döngüsünde Break Talimatı İcra Akışı



Şekil 18. For ve For-Else Döngülerinde Break Talimatı İcra Akışı

Aşağıda bir dizide beşe bölünen ilk elemanı bulan ve döngüyü sonlandıran program örneği verilmiştir.

```
dizi=[1,3,6,7,15,31,27]
for eleman in dizi:
    if eleman%5==0:
        print('Dizide 5 e bölünen ilk eleman:',eleman)
        break
```

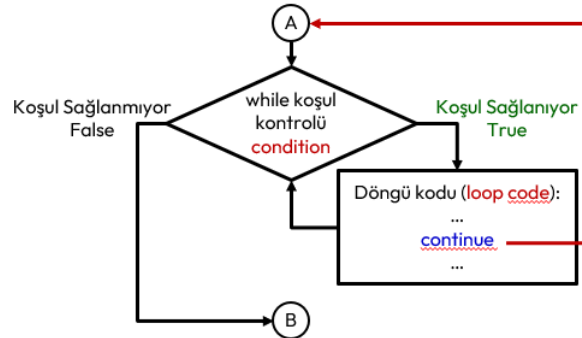
Örneğin program akışı aşağıdaki gibi olacaktır.

Dizide 5 e bölünen ilk eleman: 15

İç içe döngülerde break talimatı yalnızca içinde bulunduğu döngüyü sonlandırır!

Continue Talimatı

Döngülerde tekrarlanan kodun icrasını keserek bir sonraki elemandan döngü kodunun icrasını devam ettirmek için `continue` talimatı kullanılır. Yani geçerli döngünün başına döndürerek bir sonraki yinelemeye başlamak için kullanılır. Hem `while` hem de `for` döngülerinde kullanılabilir.



Şekil 19. While Döngüsünde Continue Talimatı İcra Akışı

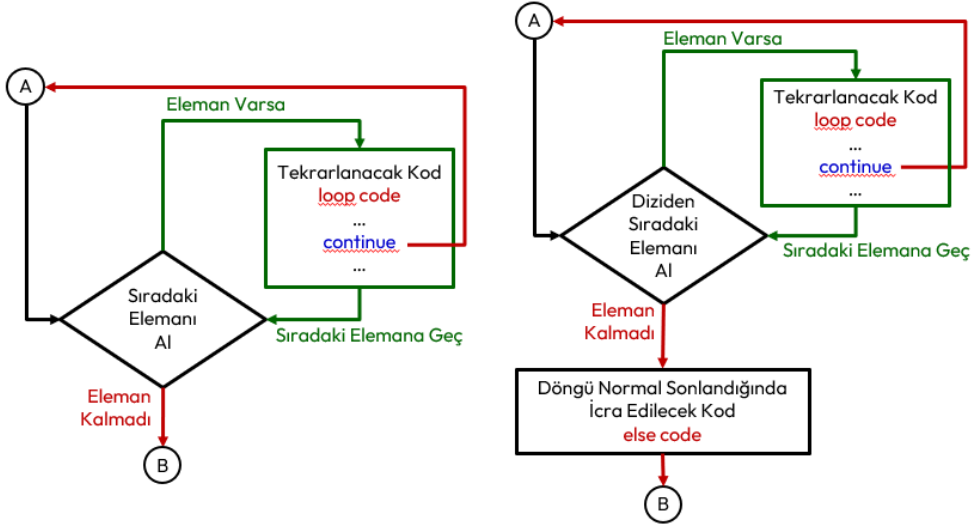
Aşağıda bir sayının asal çarpanlarını bulan programın örneği verilmiştir.

```
sayı=30
print(sayı, ' sayısının asal çarpanları:')
sayaç=2
while sayı>1:
    if sayı%sayaç==0:
        print(sayaç)
        sayı=sayı/sayaç
        continue
    sayaç+=1
```

Örneğin program akışı aşağıdaki gibi olacaktır.

30 sayısının asal çarpanları:

2
3
5



Şekil 20. For ve For-Else Döngülerinde Continue Talimatı İcra Akışı

Bir başka örnek olarak içinde 'h' harfi ayıklanmış bir metin örneği verilebilir;

```
for harf in 'Merhaba':
    if harf=='h':
        continue
    print(harf)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

M
e
r
a
b
a

Pass Talimatı

Bir talimatın gerekli olduğu ancak icra edilmesi istenmediği durumlarda `pass` talimatı kullanılır. Yani `pass` talimatı icra edildiğinde hiçbir şey olmayacağı anlamına gelir. Bu talimat, kod parçasının daha sonra ekleneceği yerlerde de kullanışlıdır.

```
for sayac in range(11):
    if sayac%2==0: #sayac çift ise
        print('{} sayısi çifttir.'.format(sayac))
    else:
        pass # tek olması halinde icra edilecek talimatlar sonra yazılacak.
```

Örneğin program akışı aşağıdaki gibi olacaktır.

0 sayısi çifttir.
2 sayısi çifttir.
4 sayısi çifttir.
6 sayısi çifttir.
8 sayısi çifttir.
10 sayısi çifttir.

Bir sonsuz döngü talimatı aşağıdaki şekilde yazılabilir. Bitirmek için `Ctrl+C` tuşuna basılmalıdır.

```
>>> while True:pass
```

Return talimatı

Bu talimat fonksiyon başlığında incelenecektir.